

Backend Explanation Document

Citizen Identity Management System API

Introduction:

The Citizen Identity Management System API is a backend service that manages citizen records and identity card applications. The goal of this system is to simulate how a national identity authority could store citizen information and process requests for identity documents. Citizens can apply for ID cards, and officers can review and approve these applications.

The backend is built using the Python framework FastAPI and the database used is MySQL. The API follows REST principles, which means it uses standard HTTP methods such as GET, POST, PATCH, and DELETE. All responses are returned in JSON format.

The system supports multiple roles including citizens, registration officers, verification officers, and administrators. Each role has specific permissions in the system.

System Architecture:

The backend system is organized into multiple layers. The first layer is the API layer, which receives requests from clients. These requests are sent to specific routes that handle different tasks such as login, application submission, or retrieving data.

The second layer is the authentication and authorization layer. This layer checks whether the user is logged in and whether they have permission to perform the requested action.

The third layer contains the business logic. This layer processes the request and performs operations such as validating data or processing an application.

The final layer is the database layer. This layer executes SQL queries and stores or retrieves data from the database.

When a request is received, the system first checks authentication, then verifies permissions, executes the necessary database queries, and finally sends a response back to the client.

API Design:

The API follows the REST architecture style. All endpoints are organized under the base path `/api/v1/`. This versioning allows the API to be updated in the future without breaking existing clients.

Different HTTP methods are used depending on the type of operation. GET is used for retrieving information, POST is used for creating new records, and PATCH or PUT is used for updating data.

The API also returns proper HTTP status codes. For example, a successful request returns a success code, while invalid login attempts return an unauthorized error. This helps clients understand the result of each request.

Authentication:

Authentication in the system is implemented using JSON Web Tokens (JWT). When a user logs in, they provide an email and password. The system searches the Users table and verifies the password using the hashing algorithm bcrypt.

Passwords are never stored in plain text. Instead, they are stored as secure hashes in the database.

If the login is successful, the system creates a JWT token. This token contains information about the user such as their role and CNIC. The token must be included in future requests so the system can verify the user's identity.

Role Based Access Control:

The system uses role-based access control to manage permissions. Each user has a role stored in the Users table. The main roles in the system are citizen, registration officer, verification officer, and admin.

Citizens can submit identity card applications and view their status. Officers can review and process applications. Administrators have higher privileges and can manage system data and officer accounts.

Before performing any protected operation, the system checks the user's role from the JWT token. If the user does not have permission for the requested action, the system returns a forbidden error.

Database Design:

The database is designed using a relational model. The system contains multiple tables that represent different entities such as persons, citizens, officers, addresses, cities, provinces, applications, payments, and identity cards.

Each table has a primary key, and relationships between tables are enforced using foreign keys. This ensures that data remains consistent and valid.

For example, the Citizen table references the Person table, and the Application table references the Citizen table. These relationships help maintain a structured database design.

The database also uses constraints to ensure valid data. For example, an identity card expiry date must be later than the issue date, and address validity dates must follow a logical order.

Schema Changes:

Some changes were made to the database schema during the backend implementation phase.

One important change was the addition of the Users table. This table stores login credentials such as email, password hash, role, and CNIC. It is used for authentication and authorization.

Previously, the Officer table contained email and password fields. These fields were removed to avoid duplication and to follow better database design practices. All authentication data is now stored only in the Users table.

Because of this change, the seed data was also updated. Officer records now only contain operational data such as employee ID, branch information, and phone numbers. Separate user accounts were created in the Users table for officers and administrators.

Transaction Management:

Transaction management is used in the backend to ensure that important operations are completed safely. A transaction groups multiple database operations together so they either all succeed or all fail.

Transactions are implemented using the SQL commands BEGIN, COMMIT, and ROLLBACK.

One example of transaction usage is when a citizen submits an application. The system creates an application record and a payment record. If the payment creation fails, the system rolls back the entire transaction so the application is not saved without payment information.

Another example occurs when an officer approves an application. The system generates a new identity card record and updates the application status. If the identity card creation fails, the transaction is rolled back to prevent inconsistent data. Screenshots or logs showing rollback behavior are included in the project submission.

Raw SQL Usage:

The backend uses raw SQL queries instead of relying entirely on an ORM. This provides better control over database operations and allows explicit management of transactions.

All queries use parameterized inputs to prevent SQL injection attacks. This ensures that user input is handled safely and cannot modify SQL commands.

API Documentation:

The API includes interactive documentation generated by FastAPI. Developers can access this documentation through the built-in Swagger interface. The

documentation shows all endpoints, request parameters, response formats, and authentication requirements.

The OpenAPI specification file is also exported and included in the project submission as swagger.json.

Capabilities of the current API:

The API provides several important features for managing citizen identity data. Users can log in securely and receive authentication tokens. Citizens can submit identity card applications and check their status. Officers can review applications and approve or reject them.

The system can generate identity cards after approval and track payment information related to applications. It also stores detailed personal records, including family relationships, marriage records, and address history.

Overall, the system provides a structured and secure backend for managing citizen identity information and processing identity card applications.